PRIVOKE: A Client-Side Semantic Privacy Firewall for User Prompts in Web-Hosted LLMs

Nicholas Bisset*
Monash University Clayton, Australia
nbis0006@student.monash.edu

Manpreet Sahadevan, Nixon Ngai, and Sanoop Mallissery
Monash University Malaysia, Malaysia
{msah0018, nnga0004}@student.monash.edu, sanoop.mallissery@monash.edu

***Abstract*—Web-hosted language models rely on user-supplied context, but the same information that makes a response useful may disclose personal or organizational information to an external provider. We present PRIVOKE, an implemented client-side privacy firewall that inspects supported prompts locally and applies an allow, warn or block decision before transmission. A separate training service computes detector updates from supplied labelled examples, checks their version and retention constraints, and distributes accepted parameters to clients without an application release. Locally randomized telemetry follows an independent reporting path and does not supply training data. We evaluate the system through three questions concerning detection and enforcement, retention during detector updates, and systems and privacy overhead. Updated packages improved agreement on an authored contextual workload, but action failures remained and specificity on a separate annotation-presence task was only 13.36 to 21.76%. In a matched continuation study, every full-model candidate failed the retention guard despite improving aggregate agreement. Installed-browser probes confirmed the declared request behavior on the tested paths, while cost measurements and telemetry audits exposed limits in attribution and estimation. The results establish an auditable protection and update lifecycle, with independent contextual accuracy, useful adaptation and practical user benefit still requiring further evidence.**

***Index Terms*—Prompt privacy, local inference, request enforcement, guarded model updates, local differential privacy.**

I. INTRODUCTION

Web-hosted large language models support writing, coding and analysis by processing the instructions and contextual material supplied in a prompt [1]–[3]. In practical use, this context may include a person's health information, a colleague's details or confidential organizational material. Users can therefore disclose sensitive information while pursuing an otherwise legitimate task, as reported workplace incidents illustrate [4], [5]. Provider-side policies and controls influence how submitted information is handled, but they operate after the content has reached the provider [6]–[8]. Inspecting an outgoing request on the user's device creates an earlier opportunity to intervene, before that disclosure occurs.

This opportunity introduces several distinct technical questions. Recognizing a name or identifier does not establish whether its disclosure is authorized, because the decision also depends on the sensitivity of the information and its intended audience. A detector's output must then produce an observable request action: displaying a warning and preventing transmission have different consequences. Maintaining the detector adds a further difficulty, since an update that improves average accuracy can lose protection on a previously correct case. Finally, collecting information about these decisions creates its own disclosure channel. PRIVOKE addresses these concerns through a common system design that connects local inspection and explicit enforcement with guarded updates and separately randomized telemetry.

PRIVOKE represents a prompt using contextual sensitivity, intended audience and information categories, and maps those findings to ALLOW, WARN or BLOCK. The extension forwards the original request body for ALLOW and WARN, whereas BLOCK cancels a supported request. WARN attempts to display a notice but does not wait for acknowledgement, so its presence establishes neither consent nor prevented disclosure. In the configuration evaluated here, prompt inspection runs on the workstation. A distinct server-side runtime trains candidates from supplied labelled examples, and an update service validates each candidate against its exact base before publishing accepted parameters. Clients verify the distributed artifact and refresh their detector without rebuilding the extension. Neither intercepted user prompts nor telemetry supplies training labels.

Local prompt protection has substantial prior precedent. Casper combines rules, entity recognition and topic detection in a browser extension [9], [10]; Hide and Seek trains local models to hide and recover sensitive information [11]; and ProSan trains a local sanitizer to balance privacy hiding with task utility [12]. Our contribution builds on these established approaches by implementing and evaluating a version-bound detector lifecycle together with explicit request actions. The evaluation follows a candidate from its fixed base through retention assessment and publication, preserving both gains and regressions. This makes the conditions for accepting a revision inspectable, while avoiding the assumption that a passing check guarantees safe adaptation beyond the cases it covers.

We organize the evaluation around three questions:

1) **RQ1: Does PriVoke detect and enforce privacy-sensitive prompt policies effectively?**
2) **RQ2: Can PriVoke update its detector without sacrificing previously protected behavior?**

*Corresponding author: Nicholas Bisset.
GitHub Repository Link

3) **RQ3: What systems and privacy overhead does PriVoke introduce?**

Together, these questions connect contextual findings to request actions, test aggregate improvement against casewise retention, and examine the resources and information released by protection itself. The following sections explain the implemented mechanisms before presenting the detection, update and overhead evidence. This organization keeps the system's operational capabilities distinct from the accuracy and utility that require empirical support.

II. Related Work and Contribution

Local protection and sanitization. Casper provides a direct precedent for layered browser-local inspection, combining rules, entity detection and topic analysis [9]. Its prototype replaces identifiers with placeholders and supports user-triggered restoration in responses, while topic warnings require acknowledgement. Users trigger analysis through a keyboard shortcut; this interaction should not be conflated with automatic interception of every outgoing request. Casper also permits custom rules and topics and compatible model replacement, so it is not accurately described as uniformly static.

Hide and Seek (HaS) modifies the prompt before cloud inference and recovers hidden response entities locally [11]. Its generative variant trains hiding and recovery models, while the label-based variant reuses an entity recognizer for hiding; trained attackers assess how much of the original text they recover. ProSan uses task-importance and privacy proxies to guide selective word replacement, then distils a sanitizer for local execution [12]. These systems establish learned local protection as well as local processing. PromptShield addresses a related enterprise problem through role/policy classification [13].

A different measured contract. PriVoke acts directly on the outgoing request: BLOCK cancels it on a supported path, while ALLOW and WARN release its original body. Masked spans returned by the detector remain evidence and are not substituted into traffic; there is also no response-recovery stage. The system's distinctive engineering focus is the lifecycle connecting a candidate tied to an exact model revision with guarded publication and verified client refresh. We evaluate that lifecycle using paired gains and losses, including candidates that the guard refuses. A separate telemetry mechanism limits the contents of randomized event reports without treating those reports as training feedback. This focus supports a bounded systems contribution, rather than a claim of uniquely local inference or superior sanitization.

Comparison boundaries. The reviewed systems measure different aspects of protection. Casper evaluates entity and topic detection; HaS measures resistance to whole-text recovery alongside downstream task proxies; ProSan measures replacement of privacy items and automatic task utility. These outcomes cannot be placed on one privacy-accuracy scale, because their targets and denominators differ. Appendix D retains the version-qualified results and interaction details. Our executed external references use Presidio with disclosed native entity-recognizer configurations [14]. They are component references, rather than reproductions of Casper, HaS or ProSan, and do not support a matched claim of system superiority.

III. System Design and Privacy Boundary

A. Scope and Trust

PriVoke inspects supported outgoing prompts to reduce inadvertent disclosure of personal, third-party or organizational information. The protection boundary includes the workstation, enabled extension, local runtime and supervisor, and authenticated model publisher. The system consequently assumes that these components operate as configured. A compromised client, a malicious publisher or page code that deliberately replaces the interception hooks lies outside that assumption; information already held by a provider is also outside the outgoing-request boundary.

The extension recognizes prompt POST formats submitted through page `fetch` and asynchronous `XMLHttpRequest`. This coverage does not automatically extend to workers, attachments, other transports or changed endpoints, each of which requires separate validation. Disabling protection bypasses inspection by design. An optional hosted-inference configuration also changes the disclosure boundary because it sends text beyond the workstation, and it is excluded from the local-inference measurements reported here.

Figure 1 separates the system into three data paths. In the first, supported prompt text passes from the browser to the workstation detector and is released to the hosted LLM only when the request action permits it. In the second, supplied labelled examples enter the training service, and accepted model parameters return to the local detector. In the third, coarse event fields are randomized locally and sent to a telemetry collector that has no training input role. Keeping these paths distinct identifies what each service receives and prevents model maintenance from requiring collection of intercepted prompts. It does not, by itself, establish that the resulting privacy decisions are correct.

B. Local Inspection and Request Enforcement

The local inspection path begins when a page hook extracts the prompt from a supported outgoing request. The extension background process passes this text through a loopback gRPC-Web bridge to the Python runtime on the workstation. A supervisor started through native messaging manages the bridge and runtime lifecycle. The server-side runtime used for training is a separate instance and is not a fallback destination for local user inference. After analysis, the returned action controls whether the hook releases or cancels the original request, as shown in Figure 2.

Before detection, the runtime normalizes the input while retaining offsets that map evidence back to the original text. Its default cascade evaluates regular-expression rules first, followed by enabled named entity recognition (NER) and the semantic classifier. A rule-based BLOCK stops the cascade before later detectors run. Otherwise, the runtime combines the

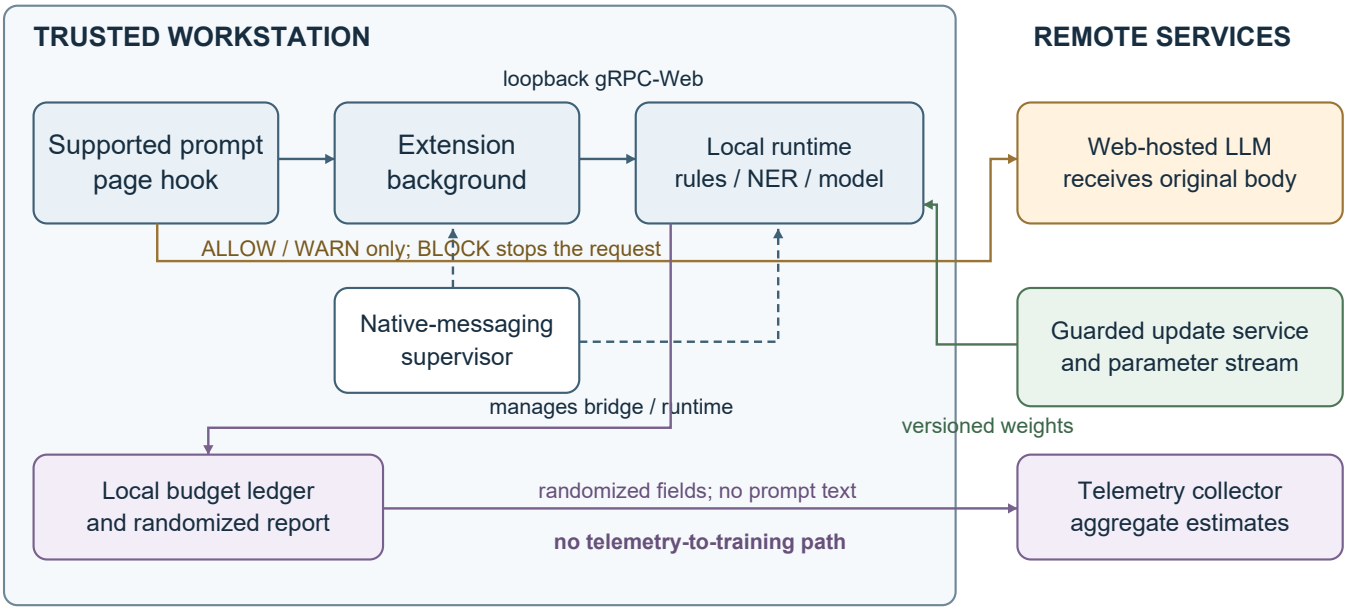


Fig. 1. PriVoke's three data paths. Supported prompt text is inspected inside the trusted workstation. Only an ALLOW or WARN decision releases the original body to the web-hosted LLM. Native messaging manages the local bridge/runtime; analysis uses loopback gRPC-Web. A separate update service supplies versioned weights, and a separate collector receives randomized event fields. Training consumes supplied labelled examples (Figure 3), not intercepted user prompts or telemetry. Dashed lines denote lifecycle control.

findings and retains the strongest action, so a less restrictive semantic prediction does not override a stronger rule finding.

Failure handling distinguishes unavailable analysis from loss of the parameter stream. A failed requested layer blocks an otherwise ALLOW result, while valid non-ALLOW findings remain visible. If analysis itself is unavailable, the extension blocks supported requests. A parameter-stream outage, however, can leave the remaining enabled rules and NER layers operating. Such fallback reduces semantic coverage and therefore need not block every full-pipeline request; the actual layers that execute are part of the evidence needed to interpret the decision.

The semantic classifier describes a prompt along three dimensions: sensitivity, ranging from none to highly sensitive; visibility, ranging from public to personal-private or unspecified; and a set of ten information categories, including health, identity and location. These dimensions distinguish the presence of information from the context in which it is disclosed. A deterministic policy then converts the labels and returned confidence into a request action. Highly sensitive content normally maps to BLOCK and potentially harmful personal information to WARN, while combinations involving identity and location can increase the restriction. Low confidence can moderate the action. These are implemented policy choices, rather than independent evidence of authorization or calibrated risk; Appendix A gives their exact mapping.

The evaluated semantic models are classifiers, rather than language models that generate conversational responses. We

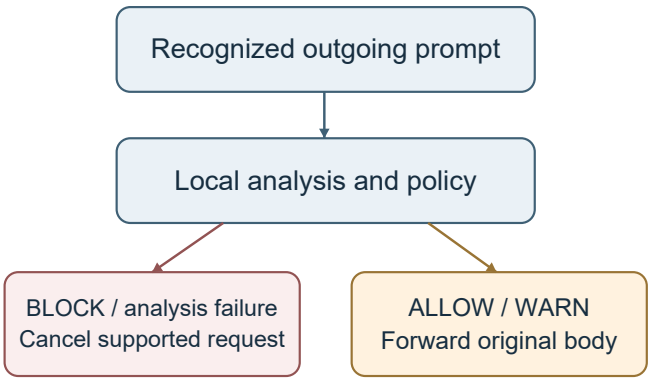


Fig. 2. Pre-transmission enforcement on supported hooks. WARN forwards unchanged content after attempting a notice; it does not wait for user acknowledgement. Analysis failure and reduced-layer stream fallback are distinct conditions. Disabled protection and unsupported traffic lie outside this decision path.

compare historical small classifiers with later packages that retain pretrained BERT encoders and poolers [15]–[18]. Each generation is offered in efficient, balanced and quality packages. The names identify configurations and do not imply a universal ordering of performance. Because tokenizers, ca-

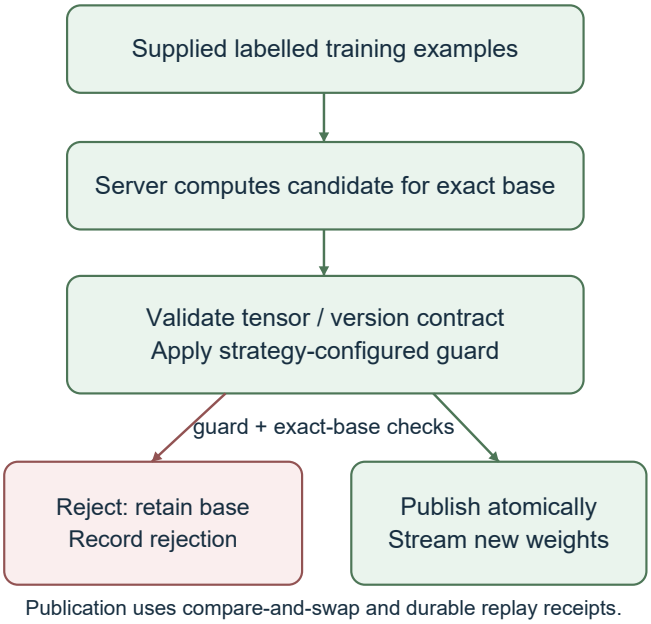


Fig. 3. A candidate is evaluated before publication. Compare-and-swap rejects stale/conflicting bases; durable receipts prevent an identical retry from applying twice. A passing retention guard covers its declared cases and criteria only. Head and underlying-model stages are separate publications, so a later rejection need not undo an earlier accepted head stage.

pacities, thresholds and training histories also differ between the native packages, their comparison measures the combined package change rather than isolating pretraining or architecture.

C. Guarded Detector Updates

The update path separates the computation of a candidate from permission to replace the model on which it was based. A training controller constructs bounded labelled batches and held-out guard cases, with protected rows, texts and groups excluded from training. The server runtime fetches the specified model, performs training and returns parameter changes and candidate metrics tied to that base. The updater then checks exact base identity, permitted trainable tensors and shapes, finite values, magnitude limits and the publication guards configured for the strategy (Figure 3). These checks serve different purposes: artifact checks establish that the update is well formed and belongs to the expected model, whereas retention checks test its behavior on declared cases. Successful computation or transport alone establishes neither accuracy nor protection.

Publication uses an atomic compare-and-swap operation: the candidate is committed only if the currently published base still matches its declared parent. Durable replay receipts bind repeated submissions to the same artifact outcome, preventing an identical retry from applying the update twice. Head and underlying-model stages remain separate publications, so rejection of a later stage does not undo an earlier accepted head update. The streaming service supplies versioned parameters, and clients verify artifact identities before refreshing their cached weights without an application release.

This implemented path supports centralized training from supplied examples while preserving local user inference. It does not collect user gradients or implement federated learning, and its existence is not evidence of cloud deployment. RQ2 therefore evaluates whether particular candidates deserve retention separately from the mechanical ability to compute, publish and receive them.

D. Separate Randomized Telemetry

The telemetry path reports five coarse event fields: action, an action/sensitivity risk bucket, one category selected by fixed precedence, a bounded release label and a four-hour time-of-day bin without a date. Each field is randomized on the workstation before release. Packets exclude raw prompts, identifiers, exact confidence scores and per-layer findings, although the retained fields still describe aspects of an event. The collector has no role in supplying corrected labels or training inputs; telemetry and detector learning remain separate processes.

For an event budget ε, each of five fields receives $\varepsilon_i = \varepsilon/5$. A field with K_i possible values reports truth with probability

$$p_i = \frac{e^{\varepsilon_i}}{e^{\varepsilon_i} + K_i - 1}, \quad q_i = \frac{1}{e^{\varepsilon_i} + K_i - 1} \tag{1}$$

for each alternative. With independent ideal samples and common public configuration, the product of the field likelihood ratios bounds an event tuple by e^{ε}. The implementation uses finite arithmetic, so agreement with this ideal expression requires a separate numerical audit. A persistent local ledger reserves budget before constructing a report, with defaults of one per event and eight per UTC day; failed construction or delivery does not refund the reservation. This accounting depends on retained state and a trusted clock. It bounds released content under its assumptions, while report presence, contribution selection, delivery and network metadata remain outside the guarantee. The collector estimates marginal counts by inverting the nominal response probabilities and clipping the results, which introduces a separate estimation cost examined in RQ3.

IV. Evaluation Method

A. Questions and Evidence Units

The evaluation treats detection, policy agreement and transmission as different outcomes, because success at one stage does not establish success at the next. Table I maps the existing experiments to the three research questions and identifies what each unit of evidence supports. Where several comparisons reuse an experiment, they do not provide independent confirmation. We also distinguish the semantic model from the complete product: semantic-only calls explicitly request and verify sole semantic execution, whereas product comparisons request rules, NER and semantics and preserve legitimate skips caused by an early rule-based block.

TABLE I
EVIDENCE UNITS AND THEIR LIMITS. PRESENCE/CATEGORY LABELS ARE AVAILABLE PROXIES; INDEPENDENTLY ADJUDICATED CONTEXTUAL PRIVACY TRUTH IS MISSING.

RQ	Evidence	What it establishes
1	1,560 authored contextual cases; 498 final presence rows; 2,000 external category rows	Agreement with distinct targets; not human privacy accuracy.
1	33 installed-browser receiver probes	Bounded request disposition; not provider coverage or usability.
2	Paired package outcomes; matched head-only/full-model continuation	Casewise gains/harms and guard decisions; not guaranteed adaptation.
3	360 browser decisions in 12 profiles; finite telemetry audit and synthetic simulation	Scoped systems cost and conditional privacy/estimation behavior.

Contextual policies. The authored workload contains 1,560 cases, of which 810 are sensitive and 750 clean, with 260 per audience class. Sensitivity, audience, category and allowed-action targets were constructed provisionally with assistant support and have not undergone independent human adjudication. They therefore support measurements of agreement with the authored policy, rather than independently established privacy or authorization accuracy. For binary scoring, nonzero sensitivity or any category defines a positive. Semantic joint correctness requires exact agreement on all three label dimensions, while product joint correctness additionally requires an allowed action. We record under-restriction when an action falls below the allowed-action set and over-restriction when it exceeds that set. Receiver-observed transmission is measured separately from both forms of policy agreement.

The workload comprises 35 scenario families and 24 audience-expression groups. It was sealed for an earlier semantic comparison, then opened and reused in the product and recognizer studies. Those later results are consequently PriVoke-authored development evidence, even when their comparison procedures were fixed in advance. Repeated passes check whether the same inputs produce the same outputs; they do not increase the independent sample. Separate paired resampling by scenario family or audience expression describes sensitivity to these dependencies without certifying population coverage.

Annotation-based checks. The once-only final endpoint contained 498 English rows from PIIMB revision `4a13e9f`, comprising 236 annotation-positive and 262 annotation-negative targets [19]. Models, source inputs and scoring were frozen before its release, and no tuning, reselection or promotion followed the results. This endpoint tests whether a prompt contains a source annotation. It is distinct both from contextual authorization and from PIIMB's official character-masking/F2 task. The 464 observed source-group keys describe grouping conventions and do not certify template independence. A separate reference used 2,000 AdvPIIBench rows: 1,000 injected positives, 750 ordinary negatives and 250 hard negatives [20]. Its official source categories are not exhaustive privacy annotations. Appendix B therefore retains the selection rules, grouping and prediction proxies needed to interpret each check.

B. Comparison and Retention Protocols

We evaluated historical and updated native packages on the same contextual cases through the product handler, holding rules and policy unchanged. This pairing exposes gains and losses on individual cases, but differences between the packages still prevent attribution to one model property. A narrower ablation removed individual detectors from the balanced package while fixing texts, weights and policy. We also ran configured small and large English Presidio models as same-text entity-recognizer references at threshold 0.5 [14], [21]. These references do not produce the contextual labels and actions required by our complete policy, so those unsupported outputs were left unmeasured.

For RQ2, a fixed-base experiment compared head-only and full-model continuation across three seeds. Each treatment began with byte-identical intact pretrained balanced encoder/pooler weights and zero task heads, received the same ordered 4,096 training exposures over 64 steps, and was assessed on the same 1,080 reused provisional cases after complete training/guard group and text exclusions. Only full-model continuation enabled encoder optimization, including the resulting changes to joint-gradient clipping. We assessed every terminal candidate, including guard failures, without using assessment results to select a checkpoint.

The 16-case retention guard required improved joint correctness and zero action harms against its derived target actions. The broader 1,080-case assessment instead permitted its declared sets of allowed actions. Thus, a guard outcome and an assessment score refer to different cases and action criteria. The small guard and weak zero-head control limit what either a pass or an improvement establishes. Appendices E and B retain the recipes and historical unsuccessful branches.

C. Enforcement, Cost and Telemetry Protocols

The installed-browser study exercised the built extension in Linux Chromium through the standard native-host installer, native messaging, supervisor, bridge and historical balanced detector. An HTTPS synthetic receiver recorded arriving original bodies and checked for late sends. Thirty-three correctness probes tested forwarding, blocking, faults, cancellation and recovery. They used the synthetic receiver rather than an authenticated LLM provider, so they establish behavior along declared request paths. Full-pipeline and sole-semantic execution were checked separately outside the resource-measurement windows.

The cost experiment requested rules and NER and measured 360 decisions across twelve fresh profiles, with two profiles for each transport/action cell. Page-observed latency ended at the analysis result and excluded the receiver response. CPU and sampled charged memory were measured for the

TABLE II
SAME 1,560 PROVISIONAL CONTEXTUAL CASES, FULL PRODUCT. EACH ENTRY IS HISTORICAL/UPDATED. JOINT AND ACTION COUNTS HAVE DENOMINATOR 1,560; SENSITIVE RECALL COUNTS 810 AND CLEAN SPECIFICITY COUNTS 750. ZERO ERRORS IN EVERY ARM.

Package	Joint	Action	Sensitive	Clean
Efficient	266/693	1029/1415	544/808	494/629
Balanced	160/717	1026/1409	771/802	318/631
Quality	226/879	1073/1421	598/810	468/641

TABLE III
FINAL SEMANTIC ANNOTATION PRESENCE: 236 POSITIVE/262 NEGATIVE ROWS. HISTORICAL/UPDATED COUNTS; TRUE/FALSE POSITIVES (TP/FP), TRUE/FALSE NEGATIVES (TN/FN). ALL SIX ARMS HAVE ZERO ERRORS.

Package	TP	TN	FP	FN
Efficient	140/218	61/45	201/217	96/18
Balanced	141/221	65/35	197/227	95/15
Quality	167/190	51/57	211/205	69/46

whole container, including resident infrastructure, cache and the observer. Because the experiment supplied neither an unprotected baseline nor attribution to individual processes, these measurements describe the instrumented system rather than incremental client overhead. They also exclude semantic-layer cost. Appendix F specifies the observation windows and sampling boundaries.

The telemetry study combined a mechanism audit with separate utility and accounting checks. The audit enumerated finite sampling kernels for one fingerprinted host build under the assumption of independent uniform OS bits. Fixed-contributor simulations measured the error introduced by estimation, while real ledger checks exercised persistence, concurrency and failure accounting. Together, these procedures examine a conditional privacy mechanism and synthetic utility; they do not constitute a deployed monitoring study.

V. RQ1: DETECTION AND ENFORCEMENT

A. Contextual Policy Agreement

The updated packages agreed more often with the authored contextual targets than their historical counterparts (Table II). All six product runs completed without failed calls or layer errors, and joint correctness increased by 27.37–41.86 percentage points. Nevertheless, exact agreement reached only 693–879 of 1,560 cases. Looking at individual cases explains why the aggregate gain is insufficient as a retention claim: efficient, balanced and quality improved on 484/573/671 joint outcomes but regressed on 57/16/18, respectively. Allowed actions improved on 389/389/348 cases and regressed on 3/6/0. Quality's zero action regressions therefore coexisted with eighteen joint-label regressions. No package was promoted.

A common restriction pattern arose from the cascade rather than differences among the updated semantic models. All updated packages over-restricted the same 139 cases. Of these, 109 clean controls received a rule-based BLOCK, which prevented later detectors from running, and thirty low-risk controls received a rule-based WARN. Across the complete workload, rules ran on every case and early-blocked 289, leaving NER and semantics to execute on 1,271. The loaded NER model produced zero converted PriVoke findings after supported-label filtering. This is a statement about the findings admitted into the policy, not evidence that the recognizer found no raw entities or was inactive.

The balanced-package ablation tested the contribution of these rules while holding the remaining configuration fixed. Removing them increased allowed-action agreement from 1,409 to 1,530 and joint agreement from 717 to 972. It corrected all 139 shared unnecessary restrictions, but also lost eighteen previously correct actions; under-restricted cases increased from twelve to thirty. Thus, the 121 net action gains conceal additional lost protection under the provisional policy targets. Removing NER left full-product outputs unchanged on this fixture, consistently with the absence of converted findings. Neither result justifies universally disabling a detector, since both depend on this workload and policy. All 10,920 ablation calls retained valid execution records.

B. What Transfers Beyond the Authored Cases?

The final annotation-presence endpoint exposed a substantially different limitation. All 2,988 semantic-only calls completed without errors and verified that only the requested layer executed; Table III retains the full confusion counts. Updated recall reached 80.51–93.64%, but specificity was only 13.36–21.76%. Relative to the historical packages, efficient and balanced lost specificity, while quality's small increase remained uncertain. Frozen source-group resampling gave paired net-accuracy gain ranges of 7.41–17.37, 5.35–15.03 and 1.02–10.71 percentage points, respectively. Improved average agreement therefore coexisted with poor discrimination of annotation-negative rows.

These were the same updated artifacts that achieved 98.40–99.87% specificity on the authored semantic-only contextual fixture. The artifact and nonzero-sensitivity-or-category response projection were held fixed; the texts and target labels changed. The contrast consequently does not show an intervening model regression, and the contributions of distribution and target differences have not been isolated. At the same time, task mismatch does not make the poor final result disappear. The two endpoints show that strong clean-control agreement on the authored task is insufficient evidence of broader discrimination, while neither supplies independently adjudicated contextual privacy truth.

The external source-category check showed another recall/specificity tradeoff under a different response proxy. Updated balanced recall was 92.9%, compared with historical 63.0%, but source-negative specificity was 0.5%, compared with 48.6%. This check counted any semantic finding as positive and used labels for injected identifier categories. Because those labels do not exhaustively describe privacy throughout the prompt, the 0.5% specificity cannot be converted into an established 99.5% broad-privacy false-alarm rate. It does show

poor agreement with the declared source target. Restricted five-type Presidio exhibited the opposite tradeoff, with 99.1% specificity and only 3.9% recall. All 6,000 model/recognizer analyses completed without errors, and no threshold or row was changed afterwards.

The same-text recognizer comparison further illustrates why model size and aggregate scores are insufficient for ranking privacy systems. On the authored contextual cases, small-English Presidio achieved 45.31% recall and 71.87% specificity. Its larger configuration reduced recall to 29.63% while increasing specificity to 88.80%. Both classified 906 of 1,560 cases correctly, but the larger model gained 192 correct cases and lost 192 others. Updated product packages achieved recall of 99.01–100.00% and specificity of 83.87–85.47% under the contextual targets. The entity/contextual task mismatch and PriVoke-authored data prevent interpreting this advantage as a general privacy ranking (Appendix C).

C. Does a Decision Control Transmission?

The completed installed-browser capture connected request decisions to receiver observations. Across thirty-three declared correctness probes, the receiver observed eighteen unchanged-body sends and fifteen zero-send outcomes. Runtime-call correlation and checks for late arrivals supported these outcomes without a decision broker, while healthy full-pipeline and sole-semantic probes verified actual execution.

The fault probes also made the limits of fallback visible. Interrupting the parameter stream blocked semantic-only requests, but the configured full-mode fallback executed rules and NER and forwarded the request. Restarting restored full execution. Disabling protection or all layers forwarded requests by design. These observations distinguish loss of semantic coverage from complete analysis failure; they do not establish that fallback preserves the original protection. The completed capture remains separate from nine earlier failed captures, which were not pooled with it.

Answer to RQ1. PriVoke detected many provisionally sensitive prompts and enforced the declared request contract on the tested installed paths. Its effectiveness is nevertheless conditional: action errors remained, WARN transmitted the original content, and separate presence/category tasks exposed weak specificity. The receiver probes establish bounded enforcement, while independent contextual accuracy, provider compatibility, prevention on every sensitive prompt and practical usability remain unestablished.

VI. RQ2: Updates and Previously Protected Behavior

A. Mechanism Versus Useful Adaptation

Audited server computation, bounded publication and client refresh established that the implemented path changes served parameters. Whether those changes improve protection is a separate empirical question. Earlier contextual, curriculum and accelerated training studies failed their declared qualification criteria, and a pretrained package that improved provisional disclosure agreement failed a serious-action veto. Appendix C retains these negative outcomes and their attempted budgets so that later gains do not obscure unsuccessful adaptation.

TABLE IV
Matched continuation on 1,080 reused provisional cases. Entries are head-only/full-model. Joint includes exact sensitivity, audience, categories and an allowed action. The unchanged zero-head base has zero joint and 480 action matches. Guard harms use sixteen separate cases.

Seed	Joint correct	Allowed action	Guard harms
42	192/257	908/993	0/1
43	204/247	906/989	1/1
44	198/268	912/1003	0/1

Later supervised packages improved authored contextual classification while retaining component and action regressions. The RQ1 comparison measures these resulting packages, not the benefit of automatic online training in isolation. Capacity, tokenizer, threshold and supervision changed together, so the observed difference cannot identify which change caused a gain. The matched continuation study addresses a narrower question by fixing the initial model and training exposure and varying whether the encoder is optimized.

B. Matched Continuation and Guard Decisions

All six 64-step fits completed with finite parameters. Head-only training preserved every encoder tensor, whereas full-model training changed 39 encoder tensors and recorded nonzero later encoder gradients. These checks confirm that the intended optimization treatments actually differed. The seven snapshots, including the unchanged base, then completed 7,560 native in-process semantic assessments without errors. Those calls repeatedly assessed the same 1,080 situations; they are not independent cases or evidence of deployed network operation.

Full-model continuation improved aggregate joint and allowed-action agreement over head-only training for each seed (Table IV). The paired results, however, show that its gains did not preserve every correct outcome. Relative to head-only training, it regressed on fourteen, twenty-six and twenty joint cases and on thirty-eight, forty-one and nineteen actions, respectively. Under-enforcement distance increased on 33/37/15 assessment cases and over-enforcement distance on 17/13/17. These directional counts describe different aspects of harm and must not be added as a mutually exclusive union.

Each full-model candidate failed the fixed retention guard because it introduced one action harm. Under that rule, the effective model would remain the unchanged base despite the candidate's better aggregate assessment. Head-only seeds 42/44 passed the guard, while seed 43 failed. These candidate-to-base mappings are hypothetical: no candidate from this experiment was published or promoted. The study therefore demonstrates that the guard detected specific losses, not that an accepted update was deployed or that the sixteen guard cases ruled out all other harms.

Answer to RQ2. PriVoke implements version-bound updates and rejects candidates that violate the configured retention test. The experiments do not establish useful improvement without sacrificed protection: every matched full-model candidate improved aggregate agreement yet failed the guard, and the broader package gains retained casewise regressions. Useful adaptation consequently depends on the quality of supervision, the representativeness of retention cases and the acceptance criteria applied to each revision.

VII. RQ3: Systems and Privacy Overhead

A. Measured Systems Cost

The installed-browser cost protocol recorded 360 rules/NER decisions across twelve fresh-profile windows. Median page-observed decision latency ranged from 27.9 to 35.0 ms across transport/action cells; each reported p99 was that cell's observed maximum. All 240 ALLOW/WARN decisions executed both rules and NER. The 120 BLOCK decisions executed rules and legitimately skipped NER, so their timings describe a shorter cascade path. These measurements end before the receiver response and do not include semantic-layer processing.

The summed observation windows covered 126.40 seconds and recorded 193.29 CPU-seconds for the container, including 0.69 observer CPU-seconds. Maximum sampled charged memory was 1,267.91 MiB. These totals include resident services and cache, the driver and the receiver, rather than only the extension and detector. A maximum sampling gap of 622.84 ms also leaves memory spikes potentially unobserved. Final-read and cleanup checks passed, but they do not resolve this attribution limit. Without an unprotected baseline or per-role measurements, the totals describe the instrumented system and cannot isolate incremental PriVoke overhead. Appendix H reports startup intervals and cell tails.

The separate product measurements recorded lower host-call medians and larger server-process peak RSS for updated packages than for historical packages. Their fixed execution order, warming, different backends and cascade shortcuts prevent interpreting the result as a controlled speedup or a model-only cost comparison (Appendix G). Training times and allocated CUDA tensor peaks have similarly bounded meanings: they do not establish comparative training speed or whole-process memory requirements.

B. Privacy Cost and Telemetry Utility

The finite-kernel audit found that maximum event privacy loss on the fingerprinted host exceeded nominal budgets 0.5, 1 and 2 by approximately $(1.294, 1.503, 1.226) \times 10^{-15}$. Although the excess is tiny, the exact nominal bound fails for the audited implementation. Eight default reservations compose to approximately $8 + 1.202 \times 10^{-14}$ under the same independent uniform-entropy assumptions. Appendix I distinguishes this finite calculation from the ideal formula and states the host assumptions. It does not certify other builds or the entropy available in deployment.

The utility simulations exposed a larger practical estimation problem. Inversion followed by clipping produced substantial bias and root-mean-square error, and the separately estimated marginals did not necessarily form a coherent total. For eleven-category reports with event budget one and eighty contributors, a true first-category count of seven had clipped bias +22.84 and RMSE 42.32 counts. Estimated marginals failed to sum to eighty in 99.99% of trials. These results describe error conditional on the synthetic contributors and estimator; they do not measure population monitoring accuracy or establish an acceptable error threshold for deployment.

Real ledger checks preserved the daily cap across persistent state and concurrent requests. Reservations were still consumed when construction or delivery failed, as required by the accounting rule. However, limiting which events contribute creates a distinct selection effect: reversing a constructed event sequence changed BLOCK participation among the first eight reports from 100% to 0%. Randomizing report contents does not correct that selection bias and does not protect whether an event participated.

Answer to RQ3. The tested configuration incurred measurable local processing and infrastructure costs, but the measurements do not isolate incremental overhead or semantic-pipeline cost. Randomized telemetry has a conditional content bound that the finite implementation narrowly exceeds, together with substantial synthetic estimation error. Neither the privacy calculation nor the utility simulation establishes useful deployment monitoring or protection of participation and network metadata.

VIII. Discussion and Limitations

A. What the System Evidence Supports

The experiments show why the implemented lifecycle requires separate measurements of detection, enforcement and update acceptance. A better semantic classifier cannot reverse a rule-based early BLOCK within the cascade, and a correct WARN still releases the original content. Similarly, improved average candidate performance does not compel publication when a retained protection fails. The design makes these relationships inspectable by tying each action to its execution path and each candidate to its versioned base. Separate randomization limits telemetry content without making it a source of training truth.

Sanitization and recovery systems address additional consequences of intervention. Casper supports acknowledged warnings and local restoration, HaS recovers hidden response entities, and ProSan evaluates task utility after selective replacement. Their functions motivate asking whether a user can still complete an authorized task after protection is applied. PriVoke's current request-action contract does not answer that question: it neither rewrites outgoing content nor restores an answer. Implementing those functions and evaluating their usefulness would require evidence beyond correct request disposition, rather than an inference from local detection alone.

B. Ground Truth and Generalization Remain Open

Independent contextual ground truth remains the largest scientific gap. The sensitivity, audience, category and action

targets used here are assistant-provisional; agreement with them does not establish disclosure authorization or human agreement. Annotation presence and injected-category labels provide additional checks, but they answer different questions and cannot replace contextual adjudication. In particular, identifying an entity, deciding whether its disclosure is appropriate, choosing an intervention and measuring task utility each require their own targets. The contrast between high authored clean specificity and poor final annotation-negative specificity shows the limits of the available evidence without justifying dismissal of either task.

Dependence and reuse further constrain generalization. Source-group keys and lexical exclusions track declared relationships, but do not certify template independence, cover paraphrases or remove unknown upstream pretraining exposure. The final endpoint is also source-imbalanced, with 308 Nemotron, 89 AI4privacy, eighty Gretel, twenty Privy and one MAPA row. These English synthetic or machine-generated workloads do not represent other languages and organizational policies. Separate cluster sensitivities remain descriptive and unadjusted, while repeated runs and three seeds do not create independent populations. Native-package differences also limit causal attribution. Because the final endpoint has now been opened, further tuning requires a new protected successor rather than reuse of its observed failures.

C. Retention, Availability and Practical Utility

A retention guard protects only the behavior represented by its cases and criteria. This limitation is particularly consequential when the guard is small and improvement is measured against a weak zero-head control. The matched experiment retained rejected candidates as observations, not as served improvements, and the earlier negative studies likewise supplied no evidence of useful automatic adaptation. Future studies need common pinned bases, independently reviewed retention cases and explicit casewise harm criteria. Failed candidates and effective artifacts should remain distinguishable so that a successful fit is not mistaken for an accepted model.

Blocking unavailable analysis imposes an availability cost as well as a protection policy. The installed Linux probes verified declared Chromium/native receiver paths using the historical detector, leaving authenticated providers, Windows, updated-model installation, workers, attachments and other transports unverified. Returned confidence is heuristic rather than calibrated joint correctness, and WARN does not wait for informed acknowledgement. More fundamentally, no study measured authorized task completion, the cost of unnecessary intervention, warning comprehension or reformulation effort. The observed request contract therefore does not yet establish practical utility or human benefit.

The current client exposes ephemeral events for notice rendering and local request disposition, which provide a starting point for observing intervention. Fifty-three synthetic host tests validated this contract, but the fields were added after the reported browser capture and have not undergone installed-browser or overhead remeasurement. Their meanings are deliberately narrow: rendering does not prove that a user understood a notice, and native handoff does not prove that a server received the request. Page scripts can also forge or suppress the events. They are neither persisted nor sent as telemetry. A utility study still requires independently specified expected actions and joined user or task outcomes, rather than treating these diagnostic events as evidence of benefit.

D. Measurement and Reproduction Boundaries

The resource and telemetry studies also require careful interpretation of their observation boundaries. Container totals include infrastructure and instrumentation, so they do not isolate incremental client overhead; rules/NER timings likewise do not measure semantic processing. Telemetry accounting assumes retained local state and a trusted clock, while the finite privacy calculation assumes independent uniform OS bits. Participation, delivery selection and metadata remain outside those bounds. The synthetic estimator study has no deployment-usefulness threshold, and its aggregate fields supply no corrected contextual labels for training.

The sealed local source/model capsule reproduced twenty-four invented semantic requests across six packages with matching identities and outcomes. This closes a bounded local replay gap using existing host dependencies, rather than establishing public Git-only installation, reproduction in a fresh environment or a rerun of protected-final data (Appendix I). Ignored pinned source and model assets and licensing constraints remain relevant to redistribution. The next evaluation steps follow from these limits: independently adjudicated contextual targets, protected generalization studies, broader receiver coverage, baseline-relative semantic cost and human utility. Those studies would test the unresolved claims directly.

IX. Conclusion

PRIVOKE implements local prompt inspection and explicit request actions with guarded, version-bound updates and separate randomized telemetry. Detection improved on authored contextual cases, but action failures and poor specificity on separate targets limited its effectiveness. Installed-browser probes verified supported request handling, including WARN's original-content forwarding. Every matched full-model candidate improved aggregate agreement yet failed retention, demonstrating why average gains are insufficient grounds for publication. Resource measurements and telemetry audits established bounded costs and conditional privacy behavior without demonstrating deployment utility. The system provides an auditable mechanism for protection and revision; independent contextual accuracy, safe automatic improvement and human benefit remain questions for further evaluation.

APPENDIX A
EXACT POLICY REPRESENTATION

The policy uses sensitivity and visibility as separate dimensions so that information content and its disclosure context remain distinguishable. Sensitivity S0–S3 denotes no identified risk, low-risk personal context, potentially harmful personal information, and highly sensitive/identifying details or secrets. Visibility P0–P4/PU denotes public, semi-public, authentication-restricted, group-private, personal-private and unspecified context. A prompt also receives a multilabel set drawn from ten categories: HEALTH, POLITICS, RELIGION, CRIMINAL, FINANCIAL, SEXUAL, CHILD, LOCATION, IDENTITY and THIRD_PARTY. More than one category may apply to the same prompt.

The deterministic mapping assigns BLOCK to S3 and WARN to S2. S0/S1 remain ALLOW unless IDENTITY and LOCATION coexist, or either category occurs with P2–P4 visibility. Returned confidence below 0.5 moderates S3 to WARN and S2 without IDENTITY/LOCATION to ALLOW; other low-confidence classifications also map to ALLOW. During aggregation, the runtime preserves individual enforcement decisions, unions the categories and retains the strongest sensitivity and known visibility. These rules specify how the implemented policy produces an action. They do not independently validate disclosure authorization or establish confidence as a calibrated probability.

APPENDIX B
DETAILED COMPARISON PROTOCOLS AND TERMINOLOGY

The following protocol names connect the main-text comparisons to their detailed repository records. Historical and updated packages correspond to V1 and AU3-selected V3. PH3 denotes fitting task heads over frozen encoders, EN3 denotes joint continuation, and AU3 adds audience-supervised rehearsal. PIPE3 identifies the full-product comparison, while HEADS/FULL identifies head-only/full-model continuation. These names describe different recipes and execution scopes within the three research questions; they do not identify additional independent studies whenever a result is reused.

A. Training Lineage and Selection

The historical online studies bound their artifacts, curricula, trainable tensors, seeds, budgets and safety/retention predicates before assessment. Accepted and rejected attempts were retained separately so that an attempted update could not be counted as a successful publication. Later paired semantic-only HEADS/FULL and representation studies also recorded their explicit execution scopes and kept publication distinct from qualification. The repository research records preserve the complete specifications and unsuccessful attempts alongside the reported outcomes.

PH3 fitted task heads over frozen pretrained encoders; EN3 continued training of the encoder, pooler and heads together; AU3 added audience supervision with rehearsal. Validation alone selected checkpoints, and the sealed comparison was not used for tuning. This sequence identifies the ancestry of the final package, but does not isolate the effect of any one stage. Native V1 and V3 differ in capacity, tokenizer, threshold and history, and their selected task epochs exclude unknown upstream pretraining. Appendix E records the recipes, selected doses and parameter counts needed to interpret that comparison.

B. Matched Pretrained Continuation

The fixed-base host experiment compared HEADS and FULL continuation for balanced seeds 42/43/44. Each treatment started from byte-identical intact official pretrained encoder/pooler weights and zero task heads, rather than either the original V1 or AU3-selected package. Before fitting, complete TRAIN/guard group and normalized-text exclusions removed 480 AU3 validation rows, leaving 1,080 reused provisional assessment cases: 570 sensitive and 510 clean, distributed across 25 families.

Both treatments received the same ordered 4,096 TRAIN exposures over 64 steps, the same sensitivity/audience/category objective and dropout-off execution. AdamW used head/encoder rates of $10^{-3}/2\times10^{-5}$, zero weight decay, clip norm one and microbatch 4/effective batch 64. FULL alone enabled encoder gradients and the encoder optimizer group, which also changed joint-gradient clipping. The preceding two-step pilot checked execution and resource readiness only; its success was not treated as evidence of a useful training outcome.

All six terminal candidates and one unchanged ORIGINAL were assessed through the native in-process handler with explicit successful sole [4] execution and immutable identities. Joint correctness in this study required an allowed action as well as exact sensitivity, audience and category set, even though only the semantic layer executed. The 1,080 cases were therefore assessed seven times; the resulting 7,560 calls are neither independent situations nor network RPC evidence. Assessment did not select a checkpoint.

The maintained 16-case guard required improved joint correctness and zero action harms against a single derived target action. The assessment instead permitted declared sets of allowed actions on 1,080 different cases. Candidate-to-ORIGINAL mappings describe what the guard rule would retain, because no publication occurred. A guard pass is consequently bounded by both its cases and its scoring rule, with further uncertainty from three seeds sharing one weak zero-head control and the same assessment.

C. Product and External-Reference Protocol

PIPE3 compared original V1 and AU3-selected V3 efficient/balanced/quality packages through the native regex-first handler and a narrow inference-only V3 adapter. Each request explicitly selected [2,3,4], without audience hints or a presence gate. The records checked model identities, actual layer status and errors, using the real installed regex and spaCy detectors. First-pass same-case accuracy was the primary outcome. Three passes per package produced 28,080 accuracy RPCs and 24

separate warmups; the later identical passes tested repeatability rather than increasing the independent sample.

The product-scoped ablation held the AU3 balanced package, rules, NER, policy and all 1,560 reused cases fixed while changing the requested detector set. Its seven configurations were regex [2], NER [3], semantic [4], nonsemantic [2,3], no NER [2,4], no regex [3,4] and full [2,3,4]. Every request was checked for actual execution, legitimate regex skips, identities and errors. This design supports a detector-removal comparison within one package, which is interpreted separately from semantic-only model evaluation. The comparison retained native V1 windows of 64/96/128 and category threshold 0.38, whereas V3 used 512 WordPieces and threshold 0.5. Three rows exceeded the V1 efficient cap: one skipped semantics after a regex decision and two used native truncation. All remained in the comparison. These differences in windows, tokenizers, capacities, thresholds and histories prevent causal attribution to architecture or pretraining.

The CPU host used Python 3.11.9, Torch 2.11.0, NumPy 1.26.4, spaCy 3.7.5 and `en_core_web_sm` 3.7.1, with four model threads. V3 ran before V1 in fixed tier order, with one loaded process at a time. Host RPC timings included product and transport overhead, while peak RSS covered the server process. Fixed ordering, different backends, warming and regex shortcuts therefore make these cost measurements descriptive rather than controlled model comparisons.

The independent recognizer reference used Presidio 2.2.363 on the same ordered texts, with spaCy 3.7.5, native recognizers and threshold 0.5 [14]. Separate fixed configurations used `en_core_web_sm` 3.7.1 and `en_core_web_lg` 3.7.1 [21]; the larger-model follow-up was frozen before prediction. Any finding counted as a positive. Only text and language entered inference, without target hints, PriVoke rules, mapping, tuning or exclusions. Because this entity-recognition task differs from contextual policy classification, severity, audience, exact category set, joint correctness and actions remained unmeasured.

Raw same-ID pairing used the existing PIPE3 first passes rather than rerunning them. The larger-reference freeze omitted an imported analysis-helper hash. Independent checks of raw confusion and pairing support the reported counts, but cannot retrospectively complete the original freeze. Direct analyzer timing was also excluded from comparison with product RPC latency because the execution scopes differ.

The studies retain source closures, package/data/recognizer hashes, raw journals and independent audits. Tracked compact aggregates and the generator reproduce the figures without prompts or weights, whereas inference reproduction additionally requires ignored pinned assets, including historical dirty source bytes. A post-hoc raw audit reconciled all 154 declared source files and eighteen model assets with their pinned historical bytes. Against clean commit `65a7c78`, 73 source files matched exactly, 55 differed only in line endings, eleven differed beyond line endings and fifteen were missing. A byte-exact local archive preserves the required closure.

Binary recall, specificity and accuracy used 2,000 paired whole-cluster draws, seed 42 and linear 95% percentile ranges, separately by family or audience expression. These calculations describe sensitivity to the declared dependencies rather than crossed-cluster independence or population coverage. Ten fixed equal-width bins compared returned semantic confidence with observed sensitivity/category exactness when exactly one scored semantic finding was emitted. This outcome-selected subset excluded regex skips and semantic executions without an emitted finding. Since the returned scalar is the maximum sensitivity/category-head probability, it is neither a presence probability nor a probability of full correctness; ECE/Brier calibration metrics cannot be inferred from it.

D. External Official-Category Reference

The prospective AdvPIIBench reference [20] froze revision `02741d9f` and selected 1,000 injected positives, 750 ordinary negatives and 250 hard negatives. Its English machine-generated single-turn data have no human annotation and label only the attacked identifier. The source therefore does not establish exhaustive five-type coverage or broad privacy-clean negatives. The evaluated target was agreement with official CATEGORY, which remains distinct from contextual privacy.

Preparation built whole-source closure over parent IDs, exact/normalized text and typed original injected values before exclusions. Selection then retained globally one row per component and excluded few-shot-safe and protected/conflicting components. Quotas and protected TRAIN/validation/bootstrap exclusions passed before inference. These checks address declared relationships, but do not certify the 2,000 selected components as independent templates.

For this reference alone, any returned semantic finding, including WARN, counted as positive. This proxy differs from the non-S0-or-category projection used elsewhere. Balanced V1/V3 retained their native settings and used explicit successful sole [4] execution. V1 truncated 477 inputs at its 96-token bound, while V3 had no over-cap input at 512; the resulting contrast cannot be attributed solely to training.

Presidio lg was separately restricted to credit card, phone, IBAN, email and US SSN at threshold 0.0, rather than the earlier all-recognizer threshold 0.5 configuration. Paired component resampling used 2,000 draws and seed 20261014. With one row per component, this was row resampling. These choices preserve the declared reference task, but neither finding presence nor CATEGORY agreement measures attacked-span recovery or human privacy accuracy.

APPENDIX C
RETAINED UPDATE AND REFERENCE DETAIL

The unsuccessful RQ2 branches followed different protocols, so their outcomes are retained separately rather than pooled into a success rate. Early contextual grids completed 126 attempts without a retained qualifier. The curriculum/representation study completed 900 live attempts and 360 offline steps without qualification, and none of nine revised realizations in the accelerated 2,016-attempt study met its

promising criterion. Disclosure joint correctness remained zero throughout that accelerated 2,016-attempt experiment. The pretrained MiniLM package improved provisional disclosure agreement but failed its serious-action veto.

Later supervision produced larger gains on the sealed AU3 semantic comparison. EN3-to-AU3 joint classification rose from 262/267/429 to 922/972/1209 of 1,560 for efficient/balanced/quality, with paired improvements/regressions of 680/20, 732/27 and 792/12. Clean matches were 738/740/749 of 750. Semantic joint correctness here excludes action, whereas product joint correctness includes it. Quality consequently retained classification regressions despite no action distance, and efficient/balanced acquired new under-restriction harms. These supervised package outcomes do not establish a benefit from automatic online training.

For contextual Presidio, small-English TP/TN/FP/FN counts were 367/539/211/443 and large-English counts were 240/666/84/570, both with native recognizers at threshold 0.5. Updated product efficient/balanced/quality packages had paired binary improvements/regressions of 612/81, 607/80 and 613/68 against the small reference. The direction of comparison differed on a historical 502-row development annotation-presence task: original-product recall/specificity was 93.56/19.75%, compared with configured Presidio at 78.79/79.41%. Those measurements concern different targets and snapshots, and are not pooled with the contextual comparison.

On the external source-category task, balanced V1/V3 produced TP counts of 630/929 among 1,000 positives and TN counts of 486/5 among 1,000 negatives. The negative strata comprised ordinary TN 321/5 of 750 and hard TN 165/0 of 250. Five-type Presidio returned TP 39 and TN 991, comprising ordinary 750 and hard 241. Paired V3-minus-V1 ranges were +26.56–33.16 percentage points for recall and -51.13 to -45.21 for specificity, with both classes defined in every resample. These contrasts remain limited by source categories and the any-finding prediction proxy, which do not establish broad-privacy accuracy.

The updated product packages also retained substantial errors within individual label dimensions. Audience exactness was 1089/1092/1217 of 1,560, category exactness was 885/914/979, and category macro F1 was .664/.703/.761. Joint correctness among category-bearing cases was 138/149/237 of 720, while multilabel joint correctness was 51/69/125 of 456. Correct unspecified-audience labels decreased by 56/65/21 against the historical packages; efficient unspecified-audience joint correctness fell from 102 to 98 of 260. These casewise losses coexist with the overall gains. Repeated fixed-input passes produced identical classifications, actions, evidence and traces.

Post-hoc resampling by family yielded product binary-accuracy gain ranges of 21.64–29.37, 18.28–25.69 and 19.90–29.37 percentage points. Resampling by expression group yielded ranges of 23.40–28.08, 19.17–24.81 and 22.44–26.92. The separate analyses retain different dependence assumptions and describe conditional sensitivity; they do not provide independent generalization evidence.

Appendix D
Related-System Interaction Detail

TABLE V
INTERACTION FUNCTIONS REPORTED IN THE REVIEWED VERSIONS. "NOT DESCRIBED" IS LIMITED TO THE CITED VERSION AND REVIEW SCOPE.

System	Prompt and response handling	Warning, acknowledgement and request effect
Casper v1 [9]	Replaces identifiers and restores them in responses.	Topic warning has an acknowledgement control; user can inspect or restore.
HaS v1 [11]	Hides entities and locally restores response entities.	Browser warning acknowledgement and request blocking not described.
ProSan v1 [12]	Selectively rewrites prompt words; evaluates downstream utility proxies.	Browser warning acknowledgement and request blocking not described.
PriVoke	ALLOW/WARN forward original body; returned masked spans are not substituted.	WARN notice is not awaited; BLOCK cancels supported requests.

For ProSan's occupation, race and orientation items, the reviewed v1 further requires WordNet similarity below 0.9. Although this criterion constrains the replacements, it does not make the item replacement rate a prompt-level privacy metric.

Appendix E
Selected Training Lineage

PH3 fitted affine privacy heads over frozen pretrained encoders/poolers using 6,240 TRAIN and 1,560 validation rows, AdamW rate 0.001 and validation-selected checkpoints. EN3 continued each selected parent by fitting encoder, pooler and heads together on the same data. AU3, recorded under repository protocol `ba7f3990`, added 3,120 audience rows to unchanged 6,240-row PH3 rehearsal and used new 1,560-row validation.

The EN3/AU3 recipes used CPU-four-thread float32 execution, encoder/pooler rate 2×10^{-5}, head rate 10^{-4}, weight decay 0.01, dropout 0.1 and microbatch 16/effective batch 64. Loss for the final incomplete batch used its actual denominator. AU3 warmed up for 147 updates, allowed 5–30 epochs and stopped after five validation epochs without strict 10^{-4} loss improvement. Validation alone selected trained checkpoints; the sealed comparisons selected no model, threshold or recipe.

Table VII records the selected ancestry of the PIPE3 packages. AU3 completed 16/11/9 epochs but selected checkpoints at 11/6/4, so discarded optimization is excluded from that ancestry. Cumulative selected encoder/pooler epochs were 30/20/8, and head epochs were 42/27/12. V1 encoders had zero optimizer-trained epochs and bootstrap heads had 220/360/520. These values describe the selected task-training

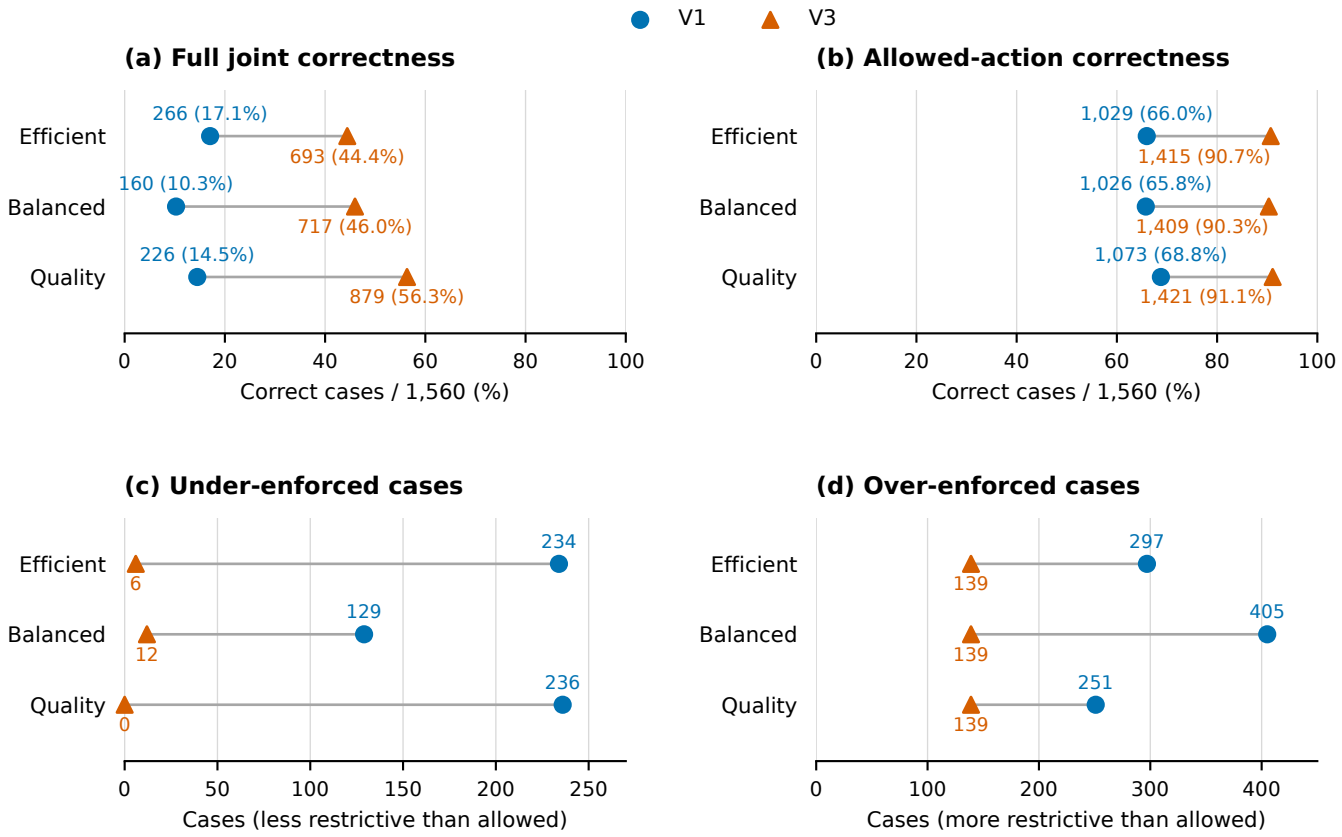


Fig. 4. Detailed product comparison on 1,560 reused provisional cases. V1 denotes historical and V3 updated packages. Under/over-enforced panels count cases on different scales, not ordinal distance; native package differences prevent causal attribution.

history; they are not equivalent to, and do not include, unknown upstream linguistic-pretraining counts.

TABLE VII
SELECTED V3 ANCESTRY. PARAMETERS ARE V1/V3 MILLIONS; PH3 IS HEAD-ONLY, EN3/AU3 JOINTLY TRAIN ENCODER, POOLER AND HEADS.

Tier	Parameters(M)	PH3	EN3	AU3
Efficient	.019/4.785	12	19	11
Balanced	.037/9.596	7	14	6
Quality	.054/17.494	4	4	4

APPENDIX F
INSTALLED-BROWSER COST PROTOCOL

The completed cost protocol used twelve fresh profiles, two for each transport/action cell. Each profile had one first decision, five warmups and thirty measured requests selecting regex/NER [2,3], with regex BLOCK permitted to skip NER. All thirty-three correctness/fault probes ran outside the resource windows. The container exposed 24 processors without CPU or memory quotas and reused resident fixture, driver and Go infrastructure.

Observation windows began before browser launch and ended after request/readiness checks but before profile teardown. CPU therefore excluded infrastructure startup that occurred before a window, whereas memory included resident services and cache. This boundary measures instrumented fresh-profile/native startup, not startup of an otherwise empty machine, and it provides no unprotected baseline.

Decision latency ran from the page-observed analysis-request message to the analysis-result message and excluded the receiver response. Pooled nearest-rank percentiles described two session clusters; p99 was the observed maximum rather than an independently estimated tail interval. The collector requested 100-ms intervals and retained every positive actual sampling gap. CPU-counter differences covered first-to-last observations, with a final read beginning after the last request, boundary checks and stop request.

Aggregate cgroup CPU included descendants, and sampled charged memory included cache and kernel charges rather than measuring RSS/PSS [24]. Totals included Chromium, runtime, Go services, fixture, driver and observer; observer CPU was retained without subtraction. Boundary checks verified known browser/supervisor/detector identities, parent links, running status and listeners. They did not establish continuous process attribution, native ancestry or incremental firewall overhead. Cleanup was required by the protocol, and the nine earlier

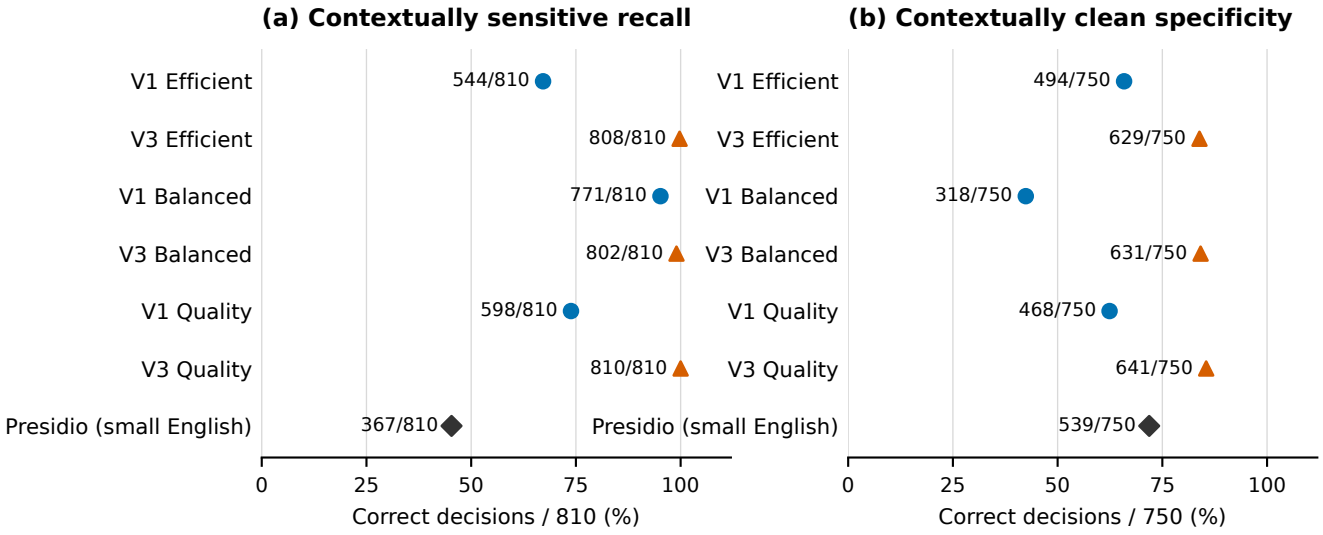


Fig. 5. Same-case binary reference: recall among 810 provisional sensitive and specificity among 750 clean cases. The diamond is small-English Presidio at threshold 0.5. PriVoke-authored reused data and entity/contextual task mismatch preclude an independent privacy ranking.

failed captures remain separate and unpooled.

Appendix G
Descriptive Product Cost

Table VIII presents the observed tradeoff between lower V3 first-pass host RPC medians and larger process peak RSS. V3 ran first, and the packages differed in NumPy/Torch backends, warming and cascade shortcuts. These conditions prevent attributing the timings to a controlled model speedup. Historical service timers and Linux lifetime-memory measurements also cover different scopes, so they cannot resolve the attribution limit or establish model-only cost.

TABLE VIII
PIPE3 DESCRIPTIVE COST: V1/V3 HOST RPC MEDIAN(P95) MS AND SERVER-PROCESS PEAK RSS MIB. EXCLUDES BROWSER OPERATION.

Tier	Median(p95), V1/V3	RSS, V1/V3
Efficient	55.20(70.08)/43.71(50.12)	389.4/466.8
Balanced	65.87(85.65)/42.73(47.82)	389.8/506.4
Quality	59.58(90.08)/50.02(59.48)	391.8/523.4

Appendix H
Installed-Browser Cost Outcomes

The completed protocol yielded twelve fresh-profile windows and 360 measured decisions (Table IX). All 240 ALLOW/WARN measurements executed regex and NER, while all 120 BLOCK measurements executed regex and legitimately skipped NER. Fresh-profile browser-context creation had nearest-rank p50 1.30 s and range 1.28–9.70; configuration/readiness transition had p50 4.37 s and range 4.21–12.21; the first post-readiness decision had p50 66.4 ms and range 53.6–75.2. These are separate instrumented intervals and should not be combined into a model-only or cold-machine startup claim.

TABLE IX
INSTRUMENTED PAGE-OBSERVED DECISION LATENCY IN MS FOR THREE FIXED SYNTHETIC CASES, REQUESTED REGEX/NER [2,3]. EACH CELL HAS SIXTY MEASUREMENTS IN TWO FRESH PROFILES; P99 IS THE OBSERVED MAXIMUM. EXCLUDES RECEIVER RESPONSE AND SEMANTIC-LAYER COST.

Transport	Action	p50	p95	p99
fetch	ALLOW	35.0	62.5	73.6
fetch	WARN	31.9	36.1	59.4
fetch	BLOCK	27.9	31.6	33.2
XHR	ALLOW	30.8	35.8	57.7
XHR	WARN	31.3	34.6	59.2
XHR	BLOCK	28.5	32.2	34.1

The twelve windows covered 126.40 s of summed spans and recorded aggregate container CPU of 193.29 s, including 0.69 s of sampled observer CPU. Maximum sampled cgroup charged memory was 1,267.91 MiB, including resident services and cache. The 1,236 samples yielded 1,224 within-window gaps, with median/p95/maximum 101.21/102.38/622.84 ms and one gap over 500 ms. Cumulative CPU is not wall time, and sampled charged-memory maxima are not process RSS/PSS; gaps also leave potential memory spikes unseen. Final-read coverage and process/listener cleanup passed, and the container exited naturally with status zero. These checks establish completion of the capture, while its whole-container scope still prevents isolation of client overhead or a speed comparison with Table VIII.

TABLE VI
EXTERNAL REPORTED OUTCOMES ON DIFFERENT TASKS; NO CROSS-STUDY RANKING OR LOCAL REPRODUCTION.

Work	Selected reported outcome	Population / boundary
Casper v1 (2024) [9]	NER recall 98.5%, specificity 86.7%	Two GPT-4-Turbo-generated 2,000-prompt workloads; NER support is 1,000 positive / 1,000 negative.
Casper 2025 [22]	PII accuracy 92.6%	Published abstract: 4,500 synthetic/2,000 real-world prompts; metric support unavailable. Separate from v1.
HaS v1 (2023) [11]	Black-box label-based protection 0.4070	19,703-example train/test corpus, 80/20; 1.7B black-box attacker; one minus whole-text recovery similarity, not entity protection; exact test support unspecified.
ProSan v1 (2024) [12]	Local-star privacy hiding rate 99.8%	MedQA, SAMSum and CodeAlpaca, 1,000 test prompts each; item-replacement proxy, exact item count unspecified.
PromptShield 2025 [23]	Policy accuracy 72.92%	Independent 435-prompt test; full model/class supports unavailable.

APPENDIX I
TELEMETRY AND LOCAL RECONSTRUCTION DETAILS

The audit enumerated exact rational finite kernels for the actual five domains, $K = 3, 4, 11, 2, 6$, on a fingerprinted Windows CPython 3.11.9 build. Outward 100-digit logarithm enclosures placed maximum event loss above nominal $\varepsilon = 0.5, 1, 2$ by approximately 1.294×10^{-15}, 1.503×10^{-15} and 1.226×10^{-15}, respectively. The excess is tiny but sufficient to invalidate the exact nominal bound in these settings. Under the same entropy assumptions, eight default reservations compose to approximately $8 + 1.202 \times 10^{-14}$. The calculation remains specific to the audited host and settings; other configurations and deployed builds are uncertified.

The utility experiment used 180 fixed-truth simulation cells with 10,000 repetitions per cell. For balanced integer category counts with $K = 11$, event $\varepsilon = 1$ and $n = 80/8000$ contributing reports, first-category truth was $7/727$, clipped bias was $+22.84/+237.50$, and RMSE was $42.32/1028.47$ counts. Clipping occurred in $85.62/28.32\%$ of trials, while independently estimated marginals failed to sum to n in $99.99/99.78\%$. The larger n pools installation-days, each subject to the default eight-reservation cap.

These errors are conditional on realized contributor counts and exclude population sampling. The simulator also uses binary64 approximations of the exact finite-kernel probabilities. Numerical moment and collector checks therefore establish consistency within the implemented estimation procedure, without specifying how much error would be acceptable for a deployment use case.

The isolated ledger checks accepted eight reservations and denied the ninth, retained the cap across process reopen, and reset it on the next UTC day. They failed closed on rollback or corruption and admitted sixteen of forty concurrent epsilon-0.5 requests. Build, queue and delivery failures also consumed their reservations. A constructed 32-event sequence with 25% BLOCK produced 100% versus 0% BLOCK among the first-eight contributors when its order was reversed. This demonstrates selection sensitivity before content randomization; it is not a measurement of real deployment monitoring accuracy.

A. Local Source/Model Reconstruction

A separately sealed 195-pin local capsule copied the 158 runtime files listed in the later FN14 freeze, additional helpers, six frozen V1/AU3 model bundles and prior invented probe receipts. This inventory is distinct from the 154-file historical clean-checkout comparison. Six fresh host servers loaded isolated source copies and replayed the same four invented sole-semantic requests per package. All 24 classifications/actions and four package identity fields matched without errors. Independent post-run checks verified pins, import origins and process cessation. The result establishes bounded local reconstruction using the maintained Python environment, not public Git-only installation, a fresh dependency environment, accuracy or a protected-final rerun.

B. Implemented Finite Kernel

The finite calculation follows the actual sampler rather than substituting the ideal randomized-response probabilities. Let $\widetilde{p}_i$ denote the binary64 truth threshold. In the audited CPython implementation, a fresh uniform 53-bit integer J selects truth when $J/2^{53} < \widetilde{p}_i$. The resulting truth probability and event loss are

$$P_i = \frac{\lceil 2^{53}\widetilde{p}_i \rceil}{2^{53}}, \quad Q_i = \frac{1 - P_i}{K_i - 1}, \qquad \varepsilon_{\text{finite}} = \sum_i \log \max\{P_i/Q_i, Q_i/P_i\}. \tag{2}$$

Alternative selection uses rejection sampling. The expression assumes independent uniform OS bits and the arithmetic of the fingerprinted host, so it does not independently certify the entropy source in deployment or the behavior of other builds.

REFERENCES

[1] B. B. Gupta, G. Akshat, V. Arya, A. Wadee, D. Alsalman, and P. Vijayakumar, "Enhancing user prompt confidentiality in large language models through advanced differential encryption," *Computers & Electrical Engineering*, vol. 116, pp. 109215–109215, 02 2024.

[2] W. Liang, Y. Zhang, M. Codreanu, J. Wang, H. Cao, and J. Zou, "The widespread adoption of large language model-assisted writing across society," *Patterns*, vol. 6, no. 12, p. 101366, 2025.

[3] T. Koraag, N. Wagner, F. Dobslaw, and L. Gren, "Comparing open-source and commercial llms for domain-specific analysis and reporting: Software engineering challenges and design trade-offs," *arXiv preprint arXiv:2509.24344*, 2025.

[4] S. Ray, "Samsung bans chatgpt among employees after sensitive code leak," *Forbes Business*, 05 2023.

[5] J. Dastin and A. Tong, "Focus: Google, one of ai's biggest backers, warns own staff about chatbots," *Reuters (Technology)*, 06 2023.

[6] T. Athey, Catalini, "The digital privacy paradox: Small money, small costs, small talk," Working Paper 23488, National Bureau of Economic Research, Cambridge, MA, 06 2017.

[7] I. Barberá, "AI Privacy Risks & Mitigations – Large Language Models (LLMs)," support pool of experts (spe) programme report, European Data Protection Board (EDPB), Mar. 2025. Document submitted February 2025, updated March 2025. [Accessed: May 28, 2026].

[8] J. King, K. Klyman, E. Capstick, T. Saade, and V. Hsieh, "User privacy and large language models: An analysis of frontier developers' privacy policies," 09 2025.

[9] C. J. Chong, C. Hou, Z. Yao, and S. M. S. Talebi, "Casper: Prompt sanitization for protecting user privacy in web-based large language models," 08 2024. Version 1, 13 August 2024; supplied 13-page PDF. Not the 2025 proceedings version.

[10] C. J. Chong, C. Hou, Z. Yao, and S. M. S. Talebi, "Casper: Prompt sanitization for protecting user privacy in web-based large language models," in *2025 IEEE 12th International Conference on Cyber Security and Cloud Computing (CSCloud)*, pp. 122–131, 2025. Related preprint: arXiv:2408.07004v1.

[11] Y. Chen, T. Li, H. Liu, and Y. Yu, "Hide and seek (has): A lightweight framework for prompt privacy protection," 09 2023.

[12] Z. Shen, Z. Xi, Y. He, W. Tong, J. Hua, and S. Zhong, "The fire thief is also the keeper: Balancing usability and privacy in prompts," 06 2024.

[13] E. Keles and S. Bahtiyar, "Promptshield: Policy-aware dlp framework for generative ai prompts," in *2025 10th International Conference on Computer Science and Engineering (UBMK)*, pp. 1537–1542, 2025. Distinct conference publication record; relationship to the 2026 thesis not established.

[14] "Presidio: Customizing nlp models." Official project documentation; accessed 14 October 2026. Local baseline versions and configuration are pinned separately.

[15] "BERT Miniatures: google/bert_uncased_L-4_H-128_A-2 model card." Pinned revision c29bee83fc7f003ac8c5e6e135529da4ecddb7c3; Apache-2.0 declaration; accessed October 12, 2026.

[16] "BERT Miniatures: google/bert_uncased_L-2_H-256_A-4 model card." Pinned revision 4e937a8675e5afd9a4836735c186ec01695bc3ea; Apache-2.0 declaration; accessed October 12, 2026.

[17] "BERT Miniatures: google/bert_uncased_L-12_H-256_A-4 model card." Pinned revision 2ab9a0f41435a4d23d4cbc11cc3e7c922545f13d; Apache-2.0 declaration; accessed October 12, 2026.

[18] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," 2019.

[19] "PIIMB: PII Masking Benchmark." Pinned Hugging Face dataset card. Revision 4a13e9f; accessed 14 October 2026. Character-level zero-shot masking benchmark; CC BY-NC 4.0.

[20] R. Arpaly and Y. Birman, "Advpiibench: The adversarial collapse of pii detection in llm interactions." https://huggingface.co/datasets/roei-ar/AdvPIIBench, 2026. Dataset. Presented as a poster at the 35th USENIX Security Symposium. Pinned revision 02741d9f; accessed 14 October 2026.

[21] Explosion, "en_core_web_lg 3.7.1 model release." https://github.com/explosion/spacy-models/releases/tag/en_core_web_lg-3.7.1. Accessed 14 October 2026. MIT license; spaCy compatibility greater than or equal to 3.7.2 and less than 3.8.0.

[22] New Jersey Institute of Technology, "Casper: Prompt sanitization for protecting user privacy in web-based large language models," 2025. Institutional abstract for the 2025 proceedings; accessed 14 October 2026; per-metric supports and full published settings unavailable.

[23] Istanbul Technical University, "Promptshield: Policy-aware dlp framework for generative ai prompts," 2025. Institutional abstract for UBMK 2025; accessed 14 October 2026; full-text model, class supports and aggregation unverified.

[24] T. Heo, "Control Group v2." The Linux Kernel documentation. https://docs.kernel.org/admin-guide/cgroup-v2.html. CPU and Memory interface definitions; accessed 15 October 2026.